



模拟业务1-机票订购系统

1. 项目概述

本项目旨在开发一款在线机票订购软件，提供用户方便快捷地查询航班信息、预订机票、支付和管理订单等功能。软件将提供用户友好的界面和稳定的性能，以满足用户对航班预订的各项需求。

2. 业务需求

- 用户能够进行航班查询，包括按航线、日期、舱位等条件筛选。
- 用户可以浏览航班详情，包括航班号、起降时间、机场信息和票价等。
- 用户能够选择并预订机票，提供乘客信息、联系方式和支付方式。
- 系统应支持多种支付方式，包括信用卡、支付宝、微信支付等。
- 用户可以查看订单状态、修改订单信息或取消订单。
- 用户能够收到预订成功的确认信息和电子机票。

3. 功能规格

- 登录/注册功能：用户可以注册新账号或使用已有账号登录系统。
- 航班查询功能：用户可以根据出发地、目的地、出发日期等条件查询航班信息。
- 航班预订功能：用户可以选择航班并填写乘客信息进行机票预订。
- 支付功能：用户可以选择支付方式进行订单支付。

- 订单管理功能：用户可以查看订单状态、修改订单信息或取消订单。

4. 非功能性需求

- 性能要求：系统应具有良好的响应速度，能够在高并发情况下保持稳定性。
- 安全要求：用户信息应进行加密存储和传输，支付过程需要采用安全的加密协议。
- 用户友好性：界面设计应简洁明了，操作流程应简单易懂，提供友好的用户提示和反馈。
- 可靠性：系统应具有良好的稳定性和可靠性，确保订单数据不丢失或错误。

5. 其他要求

- 系统应支持多种语言和时区，以满足全球用户的需求。
- 数据备份和恢复：系统需要定期进行数据备份，确保数据安全和完整性。

6. 相关文档和资源

- 机场代码表：包含国际机场的三字代码和城市信息。
- 支付接口文档：描述系统与第三方支付服务的接口规范和调用方式。

7. 系统架构设计

针对机票订购软件的具体系统架构设计，我们可以考虑以下主要组件和模块：

1. 用户界面层（UI Layer）：

- 提供用户界面，包括网页端、移动端应用和可能的桌面应用。
- 实现用户注册、登录、航班查询、预订机票、订单管理等功能。
- 通过友好的用户界面向用户展示航班信息和订单状态，提供支付接口和通知功能。

2. 应用逻辑层（Application Layer）：

- 处理用户请求和业务逻辑，对用户输入进行验证和处理。
- 包括用户身份认证、航班查询逻辑、订单管理逻辑、支付逻辑等模块。
- 实现业务逻辑的封装和复用，确保系统功能的一致性和稳定性。

3. 数据访问层（Data Access Layer）：

- 负责与数据库交互，进行数据的读取、存储和更新操作。
- 包括对航班信息、用户信息、订单信息等数据的管理和持久化。
- 通过数据访问对象（DAO）或对象关系映射（ORM）技术实现数据访问逻辑。

4. 第三方服务集成层（Third-party Service Integration Layer）：

- 集成第三方支付服务，实现支付功能，包括与支付宝、微信支付、银联等支付平台的接口对接。
- 集成短信、邮件服务，实现订单确认、提醒等通知功能。
- 与航空公司的预订系统接口对接，获取实时的航班信息和票价。

5. 安全和性能优化层（Security and Performance Optimization Layer）：

- 实现用户身份认证和权限控制，确保用户信息和交易数据的安全性。
- 进行系统性能优化，包括缓存机制的设计、数据库索引的优化、请求响应的异步处理等。
- 实现日志记录和监控功能，及时发现和处理系统异常情况。

6. 数据库层（Database Layer）：

- 存储系统的持久化数据，包括用户信息、航班信息、订单信息等。
- 可采用关系型数据库（如MySQL、PostgreSQL）或NoSQL数据库（如MongoDB）进行存储。
- 设计合适的数据库表结构和索引，以支持系统的查询和数据操作需求。

7. 通用服务层（Common Service Layer）：

- 提供通用的服务组件，如日志记录、缓存管理、异常处理等。
- 封装和管理系统中常用的功能模块，提高系统的可维护性和复用性。
- 实现系统与外部服务的集成和调用，如短信服务、邮件服务、支付服务等。

8. 项目模块设计

8.1 用户管理模块

8.1.1 用户注册

用例名称

用户注册

主要参与者

普通用户

概要

用户通过填写注册信息来创建新的账户，以便使用系统的各项功能。

前置条件

用户访问注册页面，尚未拥有系统账户。

后置条件

注册成功后，系统将发送确认邮件至用户提供的邮箱，用户需通过邮件确认完成注册。

正常流程

1. 用户访问注册页面，并点击“注册”按钮。
2. 系统显示注册表单，包括以下字段：
 - 用户名（4-20个字符，包含字母、数字、下划线，不允许使用特殊字符）
 - 邮箱地址（有效的电子邮箱格式）
 - 密码（6-20个字符，包含字母、数字、符号，至少包含一个大写字母和一个数字）
 - 确认密码（与密码字段内容一致）
3. 用户填写注册信息，并点击“注册”按钮。
4. 系统验证用户输入的信息：
 - 检查用户名是否已存在，若已存在则提示用户重新输入。
 - 检查邮箱地址是否已被注册，若已注册则提示用户重新输入。
 - 检查密码格式是否符合要求，若不符合则提示用户重新输入。
5. 若验证通过，系统创建新用户账户，并发送确认邮件至用户提供的邮箱。
6. 用户登录邮箱，点击确认邮件中的链接完成注册确认。

异常流程

1. 用户名已存在：
 - 系统提示用户选择其他用户名。
2. 邮箱地址已被注册：
 - 系统提示用户选择其他邮箱地址。
3. 密码格式不符合要求：
 - 系统提示用户重新输入密码，要求密码包含至少一个大写字母和一个数字，并符合字符长度要求。
4. 用户未输入所有必填字段：
 - 系统提示用户填写所有必填字段。
5. 用户未点击确认邮件中的链接：
 - 系统提醒用户注册需要完成邮箱确认步骤。

边界值限制

- 用户名长度：4-20个字符。
- 密码长度：6-20个字符。

- 邮箱地址长度：最多255个字符。
- 用户名、密码、邮箱地址等字段的合法字符集：字母、数字、下划线、常见符号（如!@\$%^&*()_+=[{}|;':",.<>?/），不允许使用特殊字符（如空格）。
- 密码必须包含至少一个大写字母和一个数字。
- 邮箱地址必须符合电子邮箱地址的格式要求。

8.1.2 用户登录

用例名称

用户登录

主要参与者

普通用户

概要

用户使用已注册的用户名和密码登录系统，以便访问系统的各项功能。

前置条件

用户已经完成注册，并拥有系统账户。

后置条件

用户成功登录后，将进入系统主页或之前访问的页面。

正常流程

1. 用户访问系统登录页面，并输入用户名和密码。
2. 用户点击“登录”按钮。
3. 系统验证用户输入的用户名和密码是否匹配已注册的账户信息。
4. 若验证通过，系统将用户重定向至系统主页或之前访问的页面，并显示登录成功的提示信息。
5. 用户成功登录系统，可以访问系统的各项功能。

异常流程

1. 用户名或密码错误：
 - 系统提示用户用户名或密码输入有误，请重新输入。
2. 用户未输入用户名或密码：
 - 系统提示用户用户名和密码为必填字段，请重新输入。
3. 用户连续多次输入错误密码：
 - 系统提醒用户密码输入错误次数过多，请稍后再试或重置密码。

边界值限制

- 用户名长度：4-20个字符。
- 密码长度：6-20个字符。
- 用户名、密码等字段的合法字符集：字母、数字、下划线、常见符号（如!@#\$\$%^&*()_+=[{}|;':",.<>?/)）。
- 用户名、密码区分大小写。

以上是用户登录功能的详细用例描述，包括了正常流程和异常流程，以及对输入字段的边界值限制。这些限制有助于确保用户输入的信息符合系统要求，提高系统的安全性和稳定性。

8.1.3 用户信息管理

用例名称

用户信息管理

主要参与者

普通用户

概要

用户可以在系统中查看和编辑个人信息，包括修改密码、更新联系方式等。

前置条件

用户已成功登录系统，并访问到用户信息管理页面。

后置条件

用户修改个人信息成功后，系统将保存更新后的信息并提示用户修改成功。

正常流程

1. 用户登录系统，并进入用户信息管理页面。
2. 系统显示用户当前的个人信息，包括用户名、邮箱地址、联系电话等。
3. 用户选择需要修改的信息，并进行编辑或填写新信息。
4. 用户点击“保存”按钮，提交修改后的个人信息。
5. 系统验证用户输入的信息格式是否合法，并更新用户信息。
6. 系统显示修改成功的提示信息，并展示更新后的个人信息。

异常流程

1. 用户未填写任何信息：
 - 系统提示用户至少填写一个需要修改的字段。

2. 用户输入的邮箱地址格式不正确：

- 系统提示用户邮箱地址格式不合法，请重新输入。

3. 用户输入的联系电话格式不正确：

- 系统提示用户联系电话格式不合法，请重新输入。

4. 用户修改用户名：

- 若用户修改了用户名，系统需要验证新用户名是否已存在，若已存在则提示用户选择其他用户名。

边界值限制

- 用户名长度：4-20个字符。
- 邮箱地址长度：最多255个字符。
- 联系电话长度：最多20个字符。
- 用户名、邮箱地址、联系电话等字段的合法字符集：字母、数字、下划线、常见符号（如!@#\$%^&*()_+=[{}]|;':",.<>?/）。
- 邮箱地址需符合电子邮箱地址的格式要求。
- 联系电话需符合常见的电话号码格式要求。

以上是用户信息管理功能的详细用例描述，包括了正常流程和异常流程，以及对输入字段的边界值限制。这些限制有助于确保用户输入的信息符合系统要求，提高系统的安全性和稳定性。

8.2 航班查询模块

8.2.1 航班查询

用例名称

航班查询

主要参与者

普通用户

概要

用户通过输入出发地、目的地、出发日期等条件来查询符合条件的航班信息。

前置条件

用户已成功登录系统，并进入航班查询页面。

后置条件

用户查询到符合条件的航班信息后，系统将显示查询结果并提供预订选项。

正常流程

1. 用户进入航班查询页面，并输入以下查询条件：
 - 出发地（必填）
 - 目的地（必填）
 - 出发日期（必填）
 - 乘客人数（可选）
 - 舱位类型（可选）
2. 用户点击“查询”按钮，提交查询请求。
3. 系统验证用户输入的查询条件的有效性，包括：
 - 出发地和目的地是否为有效的机场代码。
 - 出发日期是否为有效的日期格式，并且不早于当前日期。
4. 系统根据用户输入的条件，在数据库中检索符合条件的航班信息。
5. 系统返回查询结果列表，包括以下航班信息：
 - 航班号
 - 起降时间
 - 出发地和目的地
 - 舱位类型
 - 剩余座位数
 - 票价等
6. 用户查看查询结果，并选择需要预订的航班。

异常流程

1. 用户未填写必填字段：
 - 系统提示用户出发地、目的地和出发日期为必填字段，请重新输入。
2. 用户输入的出发日期早于当前日期：
 - 系统提示用户出发日期不能早于当前日期，请重新选择日期。
3. 用户输入的出发地或目的地不是有效的机场代码：
 - 系统提示用户输入的出发地或目的地不符合要求，请重新输入。
4. 系统未查询到符合条件的航班信息：
 - 系统提示用户未找到符合条件的航班信息，请修改查询条件重新查询。

边界值限制：

- 出发地和目的地：采用国际机场三字代码，长度为3个字符。
- 出发日期：日期格式，不早于当前日期，不晚于一年后的日期。
- 乘客人数：大于等于1的整数，最大限制根据航班的座位数而定，通常不超过乘客的最大容量。
- 舱位类型：经济舱、商务舱、头等舱等。
- 查询结果：系统能够处理大量航班信息，并且在合理的时间内返回查询结果。

以上是航班查询功能的详细用例描述，包括了正常流程和异常流程，以及对输入字段的边界值限制。这些限制有助于确保用户输入的信息符合系统要求，提高系统的安全性和稳定性。

8.3 机票预定模块

用例名称

机票预订

主要参与者

普通用户

概要

用户通过选择航班并填写乘客信息来预订机票。

前置条件

用户已成功登录系统，并已查询到符合条件的航班信息。

后置条件

用户成功预订机票后，系统将生成订单并提供订单详情。

正常流程

1. 用户选择查询到的符合条件的航班，并点击相应的“预订”按钮。
2. 系统显示航班的详细信息，并要求用户填写以下乘客信息：
 - 乘客姓名（必填）
 - 证件类型和证件号码（必填）
 - 联系电话（可选）
 - 邮箱地址（可选）
3. 用户填写完乘客信息后，点击“继续”按钮。
4. 系统验证用户输入的乘客信息的有效性，包括：
 - 乘客姓名是否为空。
 - 证件类型和证件号码是否为有效格式。

- 联系电话和邮箱地址格式是否合法。
5. 系统确认订单信息无误后，生成订单并显示订单详情，包括以下信息：
- 航班信息（航班号、起降时间、出发地和目的地）
 - 乘客信息（姓名、证件信息）
 - 票价
 - 订单号
 - 支付金额等
6. 用户确认订单信息无误，并点击“确认支付”按钮。
7. 系统跳转至支付页面，用户选择支付方式并完成支付流程。

异常流程

1. 用户未填写必填字段：
- 系统提示用户乘客姓名和证件信息为必填字段，请重新填写。
2. 用户填写的乘客信息格式不正确：
- 系统提示用户填写的乘客信息格式不合法，请重新填写。
3. 用户选择的航班已售罄：
- 系统提示用户选择的航班座位已售罄，请返回重新选择航班。
4. 用户选择的航班在填写乘客信息和提交订单过程中已被其他用户预订：
- 系统提示用户选择的航班座位已被预订，请返回重新选择航班。

边界值限制

- 乘客姓名：最大长度限制为50个字符。
- 证件号码：根据不同的证件类型，长度和格式有所不同，系统需要根据选择的证件类型进行验证。
- 联系电话：最大长度限制为20个字符，必须符合电话号码的格式要求。
- 邮箱地址：最大长度限制为255个字符，必须符合电子邮箱地址的格式要求。

以上是机票预订功能的详细用例描述，包括了正常流程和异常流程，以及对输入字段的边界值限制。这些限制有助于确保用户输入的信息符合系统要求，提高系统的安全性和稳定性。

8.4 支付模块

8.4.1 用户支付

用例名称

支付

主要参与者

普通用户、第三方支付服务提供商

概要

用户选择支付方式完成订单支付。

前置条件

用户已成功预订机票，并进入订单支付页面。

后置条件

用户成功完成支付后，系统将更新订单状态并提供支付成功的确认信息。

正常流程

1. 用户进入订单支付页面，系统显示订单详情和支付方式选择。
2. 用户选择支付方式，如支付宝、微信支付、信用卡支付等，并点击相应的支付按钮。
3. 系统跳转至支付平台页面，用户输入支付密码或其他验证信息。
4. 用户完成支付操作，支付平台返回支付结果给系统。
5. 系统接收到支付结果，验证支付是否成功。
6. 若支付成功，系统更新订单状态为已支付，并发送支付成功的确认信息给用户。
7. 用户收到支付成功的确认信息，完成支付流程。

异常流程

1. 支付过程中网络连接断开或超时：
 - 系统提示用户支付失败，请检查网络连接并重试。
2. 支付失败或支付平台返回支付异常信息：
 - 系统提示用户支付失败，可能是由于支付密码错误、账户余额不足等原因，请用户检查支付信息并重新支付。
3. 用户取消支付操作：
 - 系统返回订单详情页面，用户可以选择重新支付或取消订单。

边界值限制

- 支付金额：根据订单金额和用户支付能力确定，需考虑支付平台的支付限额。
- 支付密码：支付密码的长度和格式根据具体的支付平台要求而定，通常为6-20个字符。
- 支付方式：系统需集成支持的多种支付方式，如支付宝、微信支付、信用卡支付等。
- 支付结果：系统需能够处理支付平台返回的各种支付结果，如支付成功、支付失败、支付超时等。

8.5 订单管理模块

8.5.1 查看订单状态

用例名称

查看订单状态

主要参与者

普通用户

概要

用户可以查看自己的订单状态，包括已支付、未支付、已取消等状态。

前置条件

用户已成功登录系统，并进入订单管理页面。

后置条件

用户成功查看订单状态后，系统将显示订单当前的状态信息。

正常流程

1. 用户登录系统，并进入订单管理页面。
2. 系统显示用户的订单列表，包括订单号、航班信息、状态等。
3. 用户选择需要查看的订单，并点击相应的订单号或查看详情按钮。
4. 系统显示订单详情页面，包括以下信息：
 - 订单号
 - 航班信息（航班号、起降时间、出发地和目的地）
 - 乘客信息
 - 订单状态（已支付、未支付、已取消等）
5. 用户查看订单状态信息，确认订单当前的状态。

异常流程

1. 用户未选择需要查看的订单：
 - 系统提示用户请选择需要查看状态的订单。
2. 用户选择的订单不存在或已被删除：
 - 系统提示用户选择的订单不存在，请重新选择。

边界值限制

- 订单号：系统分配的唯一订单号，通常为一系列数字和字母的组合，长度为10-20个字符。
- 订单状态：订单可以有多种状态，如已支付、未支付、已取消等，系统需要能够处理不同状态下的订单信息。
- 订单数量：系统能够处理大量订单信息，并且在合理的时间内返回订单列表。

以上是查看订单状态功能的详细用例描述，包括了正常流程和异常流程，以及对订单号、订单状态等边界值的限制。这些限制有助于确保用户能够正确地查看订单状态信息，提高系统的可用性和用户体验。

8.5.2 修改订单信息

用例名称

修改订单信

主要参与者

普通用户

概要

用户可以修改订单信息，如乘客信息、航班选择等。

前置条件

用户已成功登录系统，并进入订单管理页面。

后置条件

用户成功修改订单信息后，系统将保存更新后的订单信息。

正常流程

1. 用户登录系统，并进入订单管理页面。
2. 系统显示用户的订单列表，包括订单号、航班信息、状态等。
3. 用户选择需要修改的订单，并点击相应的订单号或修改按钮。
4. 系统显示订单详情页面，包括以下信息：
 - 订单号
 - 航班信息（航班号、起降时间、出发地和目的地）
 - 乘客信息
 - 订单状态
5. 用户选择需要修改的订单信息，并进行编辑或填写新信息。
6. 用户点击“保存”按钮，提交修改后的订单信息。

7. 系统验证用户输入的信息格式是否合法，包括：
 - 乘客姓名是否为空。
 - 证件类型和证件号码是否为有效格式。
8. 系统确认订单信息无误后，保存更新后的订单信息。
9. 系统显示修改成功的提示信息。

异常流程

1. 用户未选择需要修改的订单：
 - 系统提示用户请选择需要修改的订单。
2. 用户输入的订单信息格式不正确：
 - 系统提示用户订单信息格式不合法，请重新填写。
3. 用户修改的订单信息与当前订单状态不匹配：
 - 系统提示用户订单状态已发生变化，无法修改订单信息。

边界值限制

- 乘客姓名：最大长度限制为50个字符。
- 证件号码：根据不同的证件类型，长度和格式有所不同，系统需要根据选择的证件类型进行验证。
- 订单状态：系统需能够处理不同状态下的订单信息，如已支付、未支付、已取消等。

以上是修改订单信息功能的详细用例描述，包括了正常流程和异常流程，以及对乘客姓名、证件号码等边界值的限制。这些限制有助于确保用户能够正确地修改订单信息，提高系统的可用性和用户体验。

8.5.3 取消订单

用例名称：

取消订单

主要参与者：

普通用户

概要：

用户可以取消已经生成但尚未完成的订单。

前置条件：

用户已成功登录系统，并进入订单管理页面。

后置条件：

用户成功取消订单后，系统将更新订单状态并提供取消成功的确认信息。

正常流程：

1. 用户登录系统，并进入订单管理页面。
2. 系统显示用户的订单列表，包括订单号、航班信息、状态等。
3. 用户选择需要取消的订单，并点击相应的订单号或取消按钮。
4. 系统显示确认取消订单的提示框，询问用户是否确认取消订单。
5. 用户点击“确认取消”按钮。
6. 系统更新订单状态为已取消，并发送取消成功的确认信息给用户。
7. 用户收到取消成功的确认信息，完成取消订单流程。

异常流程：

1. 用户未选择需要取消的订单：
 - 系统提示用户请选择需要取消的订单。
2. 用户选择的订单已经完成或已取消：
 - 系统提示用户选择的订单已经无法取消，请选择其他订单。

边界值限制：

- 订单状态：只有处于未支付状态的订单才能被取消。
- 订单数量：系统能够处理大量订单信息，并且在合理的时间内完成取消操作。

以上是取消订单功能的详细用例描述，包括了正常流程和异常流程。这些流程有助于确保用户能够正确地取消订单，提高系统的可用性和用户体验。

9. 接口设计

1. 用户管理模块接口：

- `createUser(username: string, email: string, password: string) -> userId: int`: 创建新用户。输入值包括用户名、邮箱和密码，输出值为新用户的唯一标识符 `userId`。
- `authenticateUser(username: string, password: string) -> success: bool`: 用户身份验证。输入值为用户名和密码，输出值为身份验证是否成功的布尔值 `success`。
- `updateUser(userId: int, userInfo: object) -> success: bool`: 更新用户信息。输入值为用户唯一标识符 `userId` 和要更新的用户信息 `userInfo`，输出值为更新是否成功的布尔值 `success`。

- `deleteUser(userId: int) -> success: bool`: 删除用户。输入值为用户唯一标识符userId, 输出值为删除是否成功的布尔值success。
- `getUser(userId: int) -> userInfo: object`: 获取特定用户信息。输入值为用户唯一标识符userId, 输出值为该用户的信息userInfo。
- `getUserByEmail(email: string) -> userInfo: object`: 通过邮箱获取用户信息。输入值为用户邮箱地址email, 输出值为该用户的信息userInfo。

2. 航班查询模块接口:

- `searchFlights(departure: string, destination: string, departureDate: date) -> flights: list`: 根据出发地、目的地和出发日期查询航班信息。输入值包括出发地departure、目的地destination和出发日期departureDate, 输出值为符合条件的航班信息列表flights。
- `getFlight(flightId: int) -> flightInfo: object`: 获取特定航班信息。输入值为航班唯一标识符flightId, 输出值为该航班的信息flightInfo。

3. 机票预定模块接口:

- `createOrder(userId: int, flightId: int, passengerInfo: object) -> orderId: int`: 创建新订单。输入值包括用户唯一标识符userId、航班唯一标识符flightId和乘客信息passengerInfo, 输出值为新订单的唯一标识符orderId。
- `cancelOrder(orderId: int) -> success: bool`: 取消订单。输入值为订单唯一标识符orderId, 输出值为取消是否成功的布尔值success。

4. 支付模块接口:

- `initiatePayment(orderId: int, paymentMethod: string, paymentInfo: object) -> paymentId: int`: 发起支付。输入值包括订单唯一标识符orderId、支付方式paymentMethod和支付信息paymentInfo, 输出值为支付记录的唯一标识符paymentId。
- `processPayment(orderId: int, paymentResult: object) -> success: bool`: 处理支付结果。输入值为订单唯一标识符orderId和支付结果信息paymentResult, 输出值为处理支付结果是否成功的布尔值success。

5. 订单管理模块接口:

- `getOrder(orderId: int) -> orderInfo: object`: 获取特定订单信息。输入值为订单唯一标识符orderId, 输出值为该订单的信息orderInfo。
- `updateOrder(orderId: int, orderInfo: object) -> success: bool`: 更新订单信息。输入值为订单唯一标识符orderId和要更新的订单信息orderInfo, 输出值为更新是否成功的布尔值success。

10. 数据库设计

1. 用户表 (User)

- UserId (Primary Key, int): 用户ID
- Username (varchar): 用户名
- Email (varchar): 电子邮箱
- Password (varchar): 密码 (加密存储)
- PhoneNumber (varchar): 联系电话

2. 航班表 (Flight)

- FlightId (Primary Key, int): 航班ID
- FlightNumber (varchar): 航班号
- Departure (varchar): 出发地
- Destination (varchar): 目的地
- DepartureTime (datetime): 起飞时间
- ArrivalTime (datetime): 到达时间
- AvailableSeats (int): 剩余座位数
- TicketPrice (decimal): 票价

3. 订单表 (Order)

- OrderId (Primary Key, int): 订单ID
- UserId (Foreign Key, int): 用户ID
- FlightId (Foreign Key, int): 航班ID
- PassengerName (varchar): 乘客姓名
- PassengerIDType (varchar): 证件类型
- PassengerIDNumber (varchar): 证件号码
- OrderStatus (varchar): 订单状态 (已支付、未支付、已取消等)

4. 支付记录表 (PaymentRecord)

- PaymentId (Primary Key, int): 支付记录ID
- OrderId (Foreign Key, int): 订单ID
- PaymentMethod (varchar): 支付方式 (支付宝、微信支付、信用卡支付等)
- PaymentAmount (decimal): 支付金额
- PaymentTime (datetime): 支付时间
- PaymentStatus (varchar): 支付状态 (支付成功、支付失败等)

11. 整体架构设计

整体架构设计：

- 前端界面：用户通过前端界面与系统进行交互，包括用户注册、登录、航班查询、机票预定、订单管理和支付等功能。
- 后端服务：使用后端服务实现各个模块的业务逻辑处理，包括用户管理、航班查询、订单管理和支付等功能。
- 数据库：采用关系型数据库存储用户信息、航班信息、订单信息和支付记录等数据，通过SQL语句进行数据的增删改查操作。
- 接口通信：前端通过HTTP请求与后端服务进行通信，使用RESTful API进行数据交换和业务流程控制。
- 安全机制：采用加密算法对用户密码进行加密存储，使用HTTPS协议确保数据传输的安全性，对重要操作进行权限验证和防止SQL注入等安全措施。
- 扩展性：系统采用模块化设计，各个模块之间通过接口进行通信，方便灵活扩展和维护。

12. 前后端及API设计

前端设计：

技术栈：

- 前端框架：React.js
- 状态管理：Redux
- 样式库：Bootstrap

页面设计：

1. 用户注册页面：

- 包括用户名、邮箱、密码等注册信息的输入框。
- 注册按钮，点击后向后端发送注册请求。
- 错误信息提示框，用于显示注册过程中的错误信息。

2. 用户登录页面：

- 用户名、密码输入框。
- 登录按钮，点击后向后端发送登录请求。
- 忘记密码链接，点击后跳转到密码重置页面。

3. 航班查询页面：

- 出发地、目的地、出发日期选择框。
 - 查询按钮，点击后向后端发送航班查询请求。
 - 查询结果列表，展示查询到的航班信息。
4. 订单预定页面：
- 航班信息展示区域。
 - 乘客信息输入框。
 - 预定按钮，点击后向后端发送订单创建请求。
5. 订单管理页面：
- 订单列表，展示用户的订单信息。
 - 订单详情按钮，点击后跳转到订单详情页面。
6. 支付页面：
- 支付方式选择框。
 - 支付按钮，点击后向后端发送支付请求。

前端交互：

- 使用React Router实现页面导航和路由控制。
- 使用Redux管理全局状态，包括用户登录状态和订单状态等。
- 使用axios库发送HTTP请求与后端进行通信。

7. 后端设计：

技术栈：

- 后端框架：Node.js + Express.js
- 数据库：MySQL

后端模块设计：

1. 用户管理模块：
- 负责用户注册、登录、信息更新和删除等功能。
 - 使用Express路由处理用户请求，调用相应的服务处理业务逻辑。
2. 航班查询模块：
- 负责根据出发地、目的地和出发日期查询航班信息。
 - 封装查询航班的业务逻辑，提供给控制器调用。
3. 订单管理模块：
- 负责订单的创建、取消和查询等功能。

- 与用户、航班和支付模块进行交互，维护订单信息。

4. 支付模块：

- 负责订单的支付功能，包括支付的发起和结果处理。
- 调用第三方支付接口实现支付功能，处理支付结果并更新订单状态。

数据库设计：

- 用户表（User）、航班表（Flight）、订单表（Order）和支付记录表（PaymentRecord）等。

数据库交互：

- 使用MySQL数据库存储用户信息、航班信息、订单信息和支付记录等数据。
- 使用ORM框架（如Sequelize）实现数据库操作，构建模型与数据库表的映射关系。

5. API接口设计：

用户管理接口：

- POST /api/users/register: 用户注册接口
- POST /api/users/login: 用户登录接口
- PUT /api/users/:userId/update: 更新用户信息接口
- DELETE /api/users/:userId/delete: 删除用户接口
- GET /api/users/:userId: 获取特定用户信息接口
- GET /api/users/email/:email: 通过邮箱获取用户信息接口

航班查询接口：

- GET /api/flights/search: 查询航班信息接口
- GET /api/flights/:flightId: 获取特定航班信息接口

订单管理接口：

- POST /api/orders/create: 创建订单接口
- PUT /api/orders/:orderId/update: 更新订单信息接口
- DELETE /api/orders/:orderId/cancel: 取消订单接口
- GET /api/orders/:orderId: 获取特定订单信息接口

支付接口：

- POST /api/payments/initiate: 发起支付接口
- PUT /api/payments/:paymentId/process: 处理支付结果接口

以上是该业务的前端、后端以及API接口的详细设计。在实际开发中，需要根据设计的架构和需求进一步实现和调优。

API接口设计：

用户管理接口：

- POST /api/users/register:
 - 请求参数：{ username: string, email: string, password: string }
 - 响应数据：{ userId: int }
 - 边界条件：用户名、邮箱和密码不能为空；用户名和邮箱唯一性检查。
- POST /api/users/login:
 - 请求参数：{ username: string, password: string }
 - 响应数据：{ success: bool, userId: int }
 - 边界条件：用户名和密码不能为空；用户身份验证成功。
- PUT /api/users/:userId/update:
 - 请求参数：{ userInfo: object }
 - 响应数据：{ success: bool }
 - 边界条件：用户ID不能为空；更新用户信息成功。
- DELETE /api/users/:userId/delete:
 - 请求参数：无
 - 响应数据：{ success: bool }
 - 边界条件：用户ID不能为空；删除用户成功。
- GET /api/users/:userId:
 - 请求参数：无
 - 响应数据：{ userInfo: object }
 - 边界条件：用户ID不能为空；获取用户信息成功。
- GET /api/users/email/:email:
 - 请求参数：无
 - 响应数据：{ userInfo: object }
 - 边界条件：邮箱不能为空；获取用户信息成功。

航班查询接口：

- GET /api/flights/search:

- 请求参数: { departure: string, destination: string, departureDate: date }
- 响应数据: { flights: array }
- 边界条件: 出发地、目的地和出发日期不能为空; 查询到符合条件的航班信息。
- GET /api/flights/:flightId:
 - 请求参数: 无
 - 响应数据: { flightInfo: object }
 - 边界条件: 航班ID不能为空; 获取航班信息成功。

订单管理接口:

- POST /api/orders/create:
 - 请求参数: { userId: int, flightId: int, passengerInfo: object }
 - 响应数据: { orderId: int }
 - 边界条件: 用户ID和航班ID不能为空; 创建订单成功。
- PUT /api/orders/:orderId/update:
 - 请求参数: { orderInfo: object }
 - 响应数据: { success: bool }
 - 边界条件: 订单ID不能为空; 更新订单信息成功。
- DELETE /api/orders/:orderId/cancel:
 - 请求参数: 无
 - 响应数据: { success: bool }
 - 边界条件: 订单ID不能为空; 取消订单成功。
- GET /api/orders/:orderId:
 - 请求参数: 无
 - 响应数据: { orderInfo: object }
 - 边界条件: 订单ID不能为空; 获取订单信息成功。

支付接口:

- POST /api/payments/initiate:
 - 请求参数: { orderId: int, paymentMethod: string, paymentInfo: object }
 - 响应数据: { paymentId: int }
 - 边界条件: 订单ID不能为空; 发起支付成功。
- PUT /api/payments/:paymentId/process:

- 请求参数: { paymentResult: object }
- 响应数据: { success: bool }
- 边界条件: 支付记录ID不能为空; 处理支付结果成功。